Module 6 Report: Deploy Anywhere

Docker Compose, RabbitMQ and a background worker for the Grad Café app

Mike Sasso (JHED: msasso1)

JHU EN.605.256 Modern Software Concepts in Python

October 2026

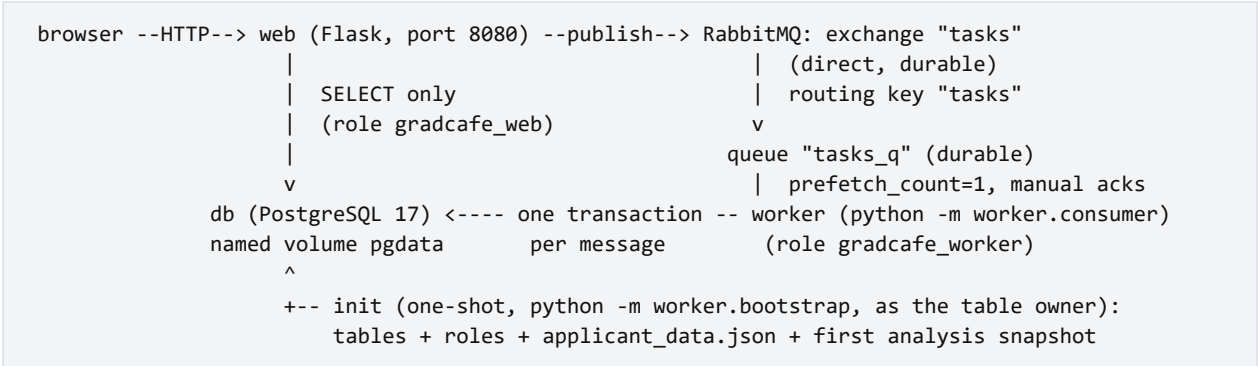
GitHub: https://github.com/sassom2112/jhu_software_concepts/tree/master/module_6 (SSH: git@github.com:sassom2112/jhu_software_concepts.git)

Docker Hub: https://hub.docker.com/r/frametotoro/module_6

1. Overview

In Module 6 I split my Module 5 Grad Café app into containers that work together. The Flask page no longer scrapes or recomputes anything while the browser waits. Its two buttons publish a task to RabbitMQ and answer HTTP 202 at once. A separate worker takes the tasks off the queue one at a time and does the work against PostgreSQL, in one transaction per task. Docker Compose builds and starts the whole stack with one command, and both service images are public on Docker Hub. Pylint still scores 10.00/10, and 528 tests cover 100% of module_6/src.

2. Architecture



Service	Image	Role in the stack	Health check
db	postgres:17	Stores the applicants, the watermark and the analysis snapshot in the named volume pgdata. Not published to the host.	pg_isready over TCP
rabbitmq	rabbitmq:4.3-management	The message broker and its management UI. Queues and persistent messages live in the volume rabbitmq_data.	rabbitmq-diagnostics -q ping
init	frametotoro/module_6:worker-v1	Runs python -m worker.bootstrap once as the table owner, then exits 0 (section 5).	-
web	frametotoro/module_6:web-v1	The Flask page and its JSON endpoints on 0.0.0.0:8080 inside the container. Reads the database; publishes tasks.	GET /healthz
worker	frametotoro/module_6:worker-v1	Consumes tasks_q and runs the two tasks (section 7).	none (RabbitMQ lists it as the consumer)

All five services share the Compose project's default network, a private bridge network, and find each other by service name (the URLs say @db:5432 and @rabbitmq:5672). Only two ports are published, both on 127.0.0.1: 8080 for the page and 15672 for the RabbitMQ UI. PostgreSQL and AMQP can only be reached inside the network. web and worker start only after init has finished successfully and RabbitMQ is healthy, so nothing runs against a half-ready database. The worker gets the data folder as a read-only mount (./src/data:/app/data:ro). Every password comes from module_6/.env, which git ignores; Compose refuses to start when one is missing instead of using an empty password.

3. Running the stack

```
git clone https://github.com/sassom2112/jhu_software_concepts.git   # or the SSH URL
cd jhu_software_concepts/module_6
cp .env.example .env          # replace every change-me (letters, digits, - and _ only)
docker compose up --build -d
docker compose ps -a          # db, rabbitmq, web healthy; worker up; init exited (0)
```

The page is at http://localhost:8080 and the RabbitMQ management UI at http://localhost:15672. RabbitMQ only accepts its built-in guest/guest account from inside its own container, so the web and worker containers could not use it. The stack creates its own broker account from RABBITMQ_DEFAULT_USER and RABBITMQ_DEFAULT_PASS in .env, and I log in to the UI with those. This is the status of my running stack:

```
SERVICE    IMAGE                           STATUS                   PORTS
db         postgres:17                     Up 2 hours (healthy)     5432/tcp
init       frametotoro/module_6:worker-v1  Exited (0) 2 hours ago
rabbitmq   rabbitmq:4.3-management          Up 2 hours (healthy)     127.0.0.1:15672->15672/tcp
web        frametotoro/module_6:web-v1     Up 2 hours (healthy)     127.0.0.1:8080->8080/tcp
worker     frametotoro/module_6:worker-v1  Up 2 hours
```

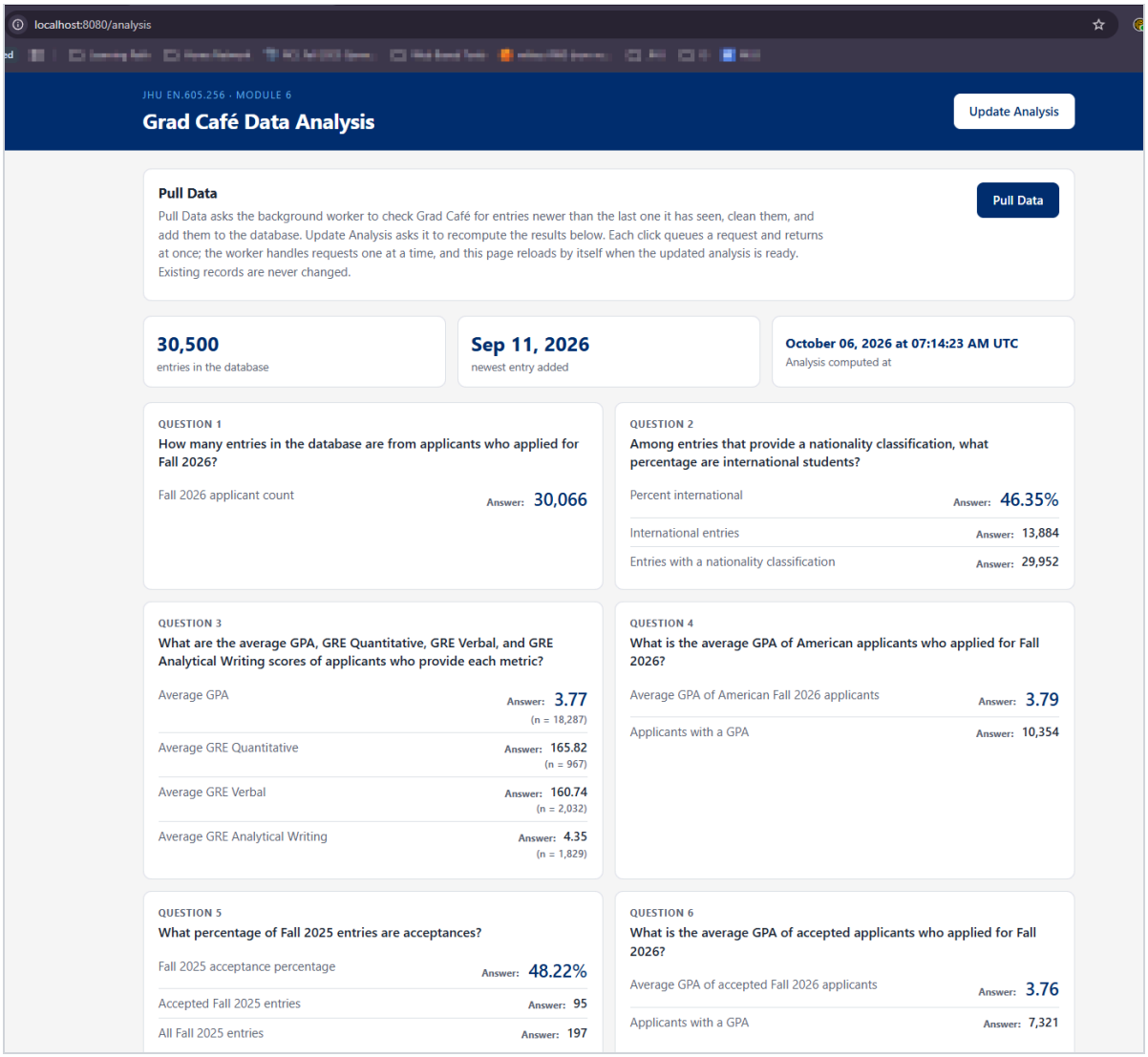


Figure 1. The running website at http://localhost:8080 after the first start: 30,500 entries and the first analysis, both from init (computed 07:14:23 UTC).

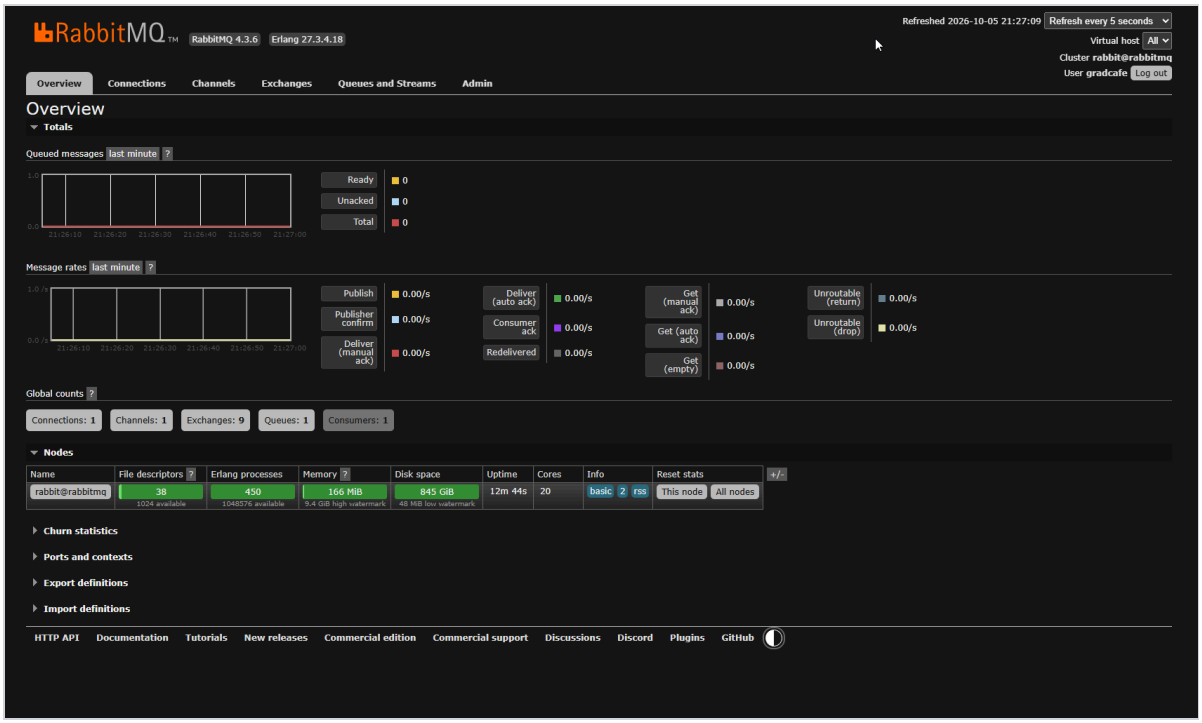


Figure 2. The RabbitMQ management UI, logged in as the account from .env: one queue and one consumer (the worker).

4. Docker images and Docker Hub

Each service has its own Dockerfile and its own requirements.txt. Both start from python:3.11-slim and install an exact, complete list of pinned packages with --no-deps, then run pip check, so the build fails if anything is missing. The build context is module_6/src, so both images can include the shared db package. The web image holds only db/ and web/: no worker code, no SQLAlchemy and no scraper libraries. Both images create a user with uid 1000 and run as USER 1000:1000, while the code stays owned by root. In Compose they also get a read-only root filesystem, no Linux capabilities and no-new-privileges.

src/web/run.py binds FLASK_HOST:PORT. Outside Docker the host defaults to 127.0.0.1, so a development server stays private. The web image sets FLASK_HOST=0.0.0.0 and PORT=8080, so inside the container Flask listens on 0.0.0.0:8080 and Docker can forward the port to it.

```
cd module_6
docker compose build                  # builds and tags both images
docker login -u frametotoro           # password: a Docker Hub access token
docker push frametotoro/module_6:web-v1
docker push frametotoro/module_6:worker-v1

# anyone can pull them:
docker pull frametotoro/module_6:web-v1
docker pull frametotoro/module_6:worker-v1
docker compose up --no-build          # runs the stack from the pulled images
```

The public repository is https://hub.docker.com/r/frametotoro/module_6. It holds both images as two tags of the same repository. The second push reused the base layers the first one had uploaded. I checked that both tags can be pulled anonymously, without logging in.

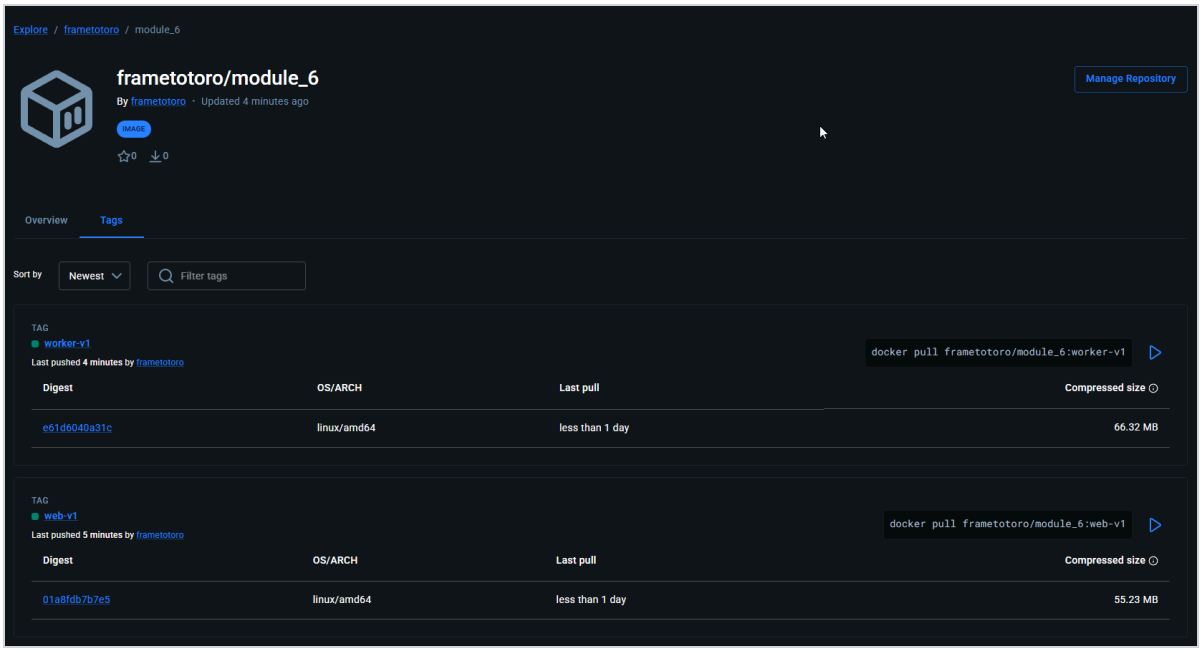


Figure 3. The public Docker Hub repository frametotoro/module_6 with the tags web-v1 and worker-v1.

5. Database initialization and least privilege

The init service runs python -m worker.bootstrap (src/worker/bootstrap.py) on every docker compose up, as gradcafe, the owner of the tables. In one transaction it:

- creates the missing tables: applicants (Module 3's table), ingestion_watermarks and analysis_snapshot (one row holding the analysis the page shows, as JSONB);
- creates or resets the two service roles (src/db/db_roles.py);
- loads src/data/applicant_data.json with src/db/load_data.py, using INSERT ... ON CONFLICT (p_id) DO NOTHING, so a second run inserts 0 rows;
- computes the first analysis snapshot when there is none yet or new rows were loaded.

The watermark table is the one the assignment gives, built with psycopg's sql module in src/db/load_data.py:

```
CREATE_WATERMARK_TABLE_STATEMENT = sql.SQL(
    "CREATE TABLE IF NOT EXISTS {table} ("
    "source TEXT PRIMARY KEY, "
    "last_seen TEXT, "
    "updated_at TIMESTAMPTZ DEFAULT now())"
).format(table=sql.Identifier(WATERMARK_TABLE))
```

Module 5's least privilege carries over and gets stricter. Only init logs in as the owner. The web app now only displays data, so its role can only read:

Role	May	May not
gradcafe_web (web)	SELECT on applicants and analysis_snapshot	write anything, or read ingestion_watermarks
gradcafe_worker (worker)	SELECT, INSERT on applicants; SELECT, INSERT, UPDATE on ingestion_watermarks and analysis_snapshot	UPDATE or DELETE applicants, TRUNCATE, DROP, ALTER, CREATE

Both roles are NOSUPERUSER, NOCREATEDB and NOCREATEROLE, have a connection limit and a statement timeout, and own nothing. All handwritten SQL is still built with psycopg's sql module, with bound parameters, allow-listed identifiers and a LIMIT on every SELECT; orm_queries.py uses SQLAlchemy's query builder, also with a LIMIT on every query.

6. RabbitMQ message flow

A click on Pull Data sends POST /pull-data, and a click on Update Analysis sends POST /update-analysis. The route calls publish_task() in src/web/publisher.py. _open_channel() reads RABBITMQ_URL, opens a BlockingConnection and a channel, declares the durable direct exchange tasks, the durable queue tasks_q and their binding with routing key tasks, and turns on publisher confirms. Declaring is idempotent: if they already exist, nothing changes.

```
def publish_task(kind: str, payload: dict | None = None, headers: dict | None = None) -> None:
    """Queue one task for the worker: *kind* names it ("scrape_new_data",
    "recompute_analytics"), *payload* carries its arguments (default {}).

    The message is persistent and confirmed by the broker.  The connection is
    always closed.  Any failure raises (see PUBLISH_ERRORS), so the caller can
    answer 503 instead of pretending the task was queued.
    """
    body = json.dumps(
        {"kind": kind, "ts": datetime.now(timezone.utc).isoformat(), "payload": payload or {}},
        separators=(",", ":"),
    ).encode("utf-8")
    conn, ch = _open_channel()
    try:
        ch.basic_publish(
            exchange=EXCHANGE,
            routing_key=ROUTING_KEY,
            body=body,
            properties=pika.BasicProperties(
                delivery_mode=PERSISTENT,
                headers=headers or {},
                content_type="application/json",
            ),
            mandatory=False,
        )
    finally:
        _close(conn)
```

The body is compact JSON with the task kind, a UTC timestamp and the payload, for example {"kind":"recompute_analytics","ts":"2026-10-05T18:00:00.123456+00:00","payload":{}}. delivery_mode=2 makes the message persistent, so a queued task survives a broker restart. The connection is closed in finally,

and any failure propagates, so the route answers 503 instead of claiming the task was queued. On success the route answers 202 at once, the page shows a "Request queued" banner, and it polls GET /api/analysis-status every 3 seconds, reloading when the analysis has a new "computed at" time.

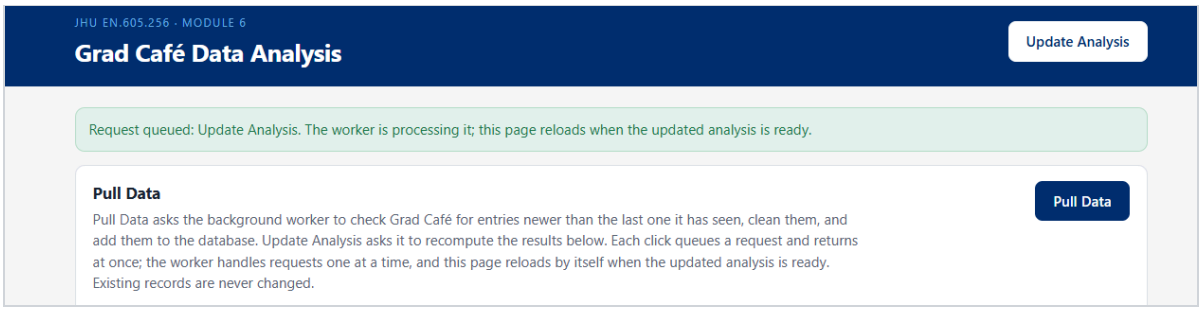


Figure 4. Update Analysis answered 202: the page shows the queued banner while the worker runs the task.

To see a message wait in the queue, I stopped the worker (docker compose stop worker) and clicked Update Analysis. RabbitMQ kept the message as Ready. When I started the worker again, it took the message, committed, acknowledged it, and the queue went back to 0.

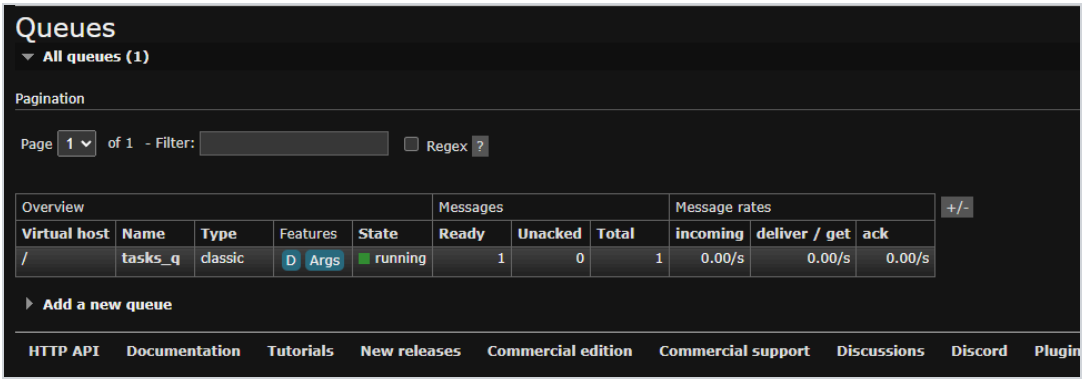


Figure 5. tasks_q with the worker stopped: one durable message waits (Ready 1).

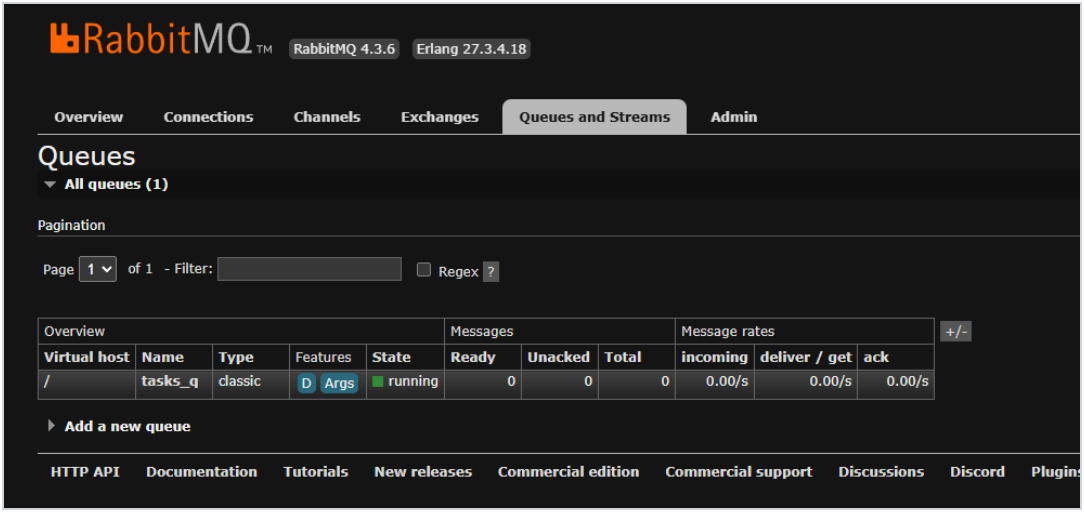


Figure 6. After docker compose start worker: the message was consumed and acknowledged (Ready 0).

7. Worker behavior

src/worker/consumer.py is a long-running process. It connects to RABBITMQ_URL, retrying up to 10 times while the broker is still starting, declares the same durable exchange, queue and binding, sets basic_qos(prefetch_count=1) and consumes with manual acknowledgements. prefetch_count=1 is the backpressure: RabbitMQ gives the worker one unacknowledged message at a time, so ten clicks wait in the queue instead of running ten scrapes at once. Each message is handled like this:

```
def process_message(self, channel, method, _properties, body: bytes) -> bool:
    """Handle one delivery completely; True if it was acknowledged, False if rejected.

    *channel* only needs basic_ack and basic_nack (a pika channel, ThreadSafeAcks or
    a test's fake); *method* carries the delivery_tag and the redelivered flag.
    Whatever the body holds, the delivery ends in exactly one ack or nack.
    """
    tag = method.delivery_tag
    kind = None
    try:
        kind, payload = parse_task(body, self.tasks)
        logger.info("delivery %s: running %s%s, payload %s", tag, kind,
                    " (redelivered)" if method.redelivered else "", _short(payload))
        started = time.monotonic()
        with self.connect() as conn, conn.transaction():
            summary = self.tasks[kind](conn, payload)
    # Whatever failed - the body, the payload, the scraper, the database, or a bug - any
    # transaction has been rolled back, and the delivery must still be answered: an
    # unanswered message would hold the worker's only prefetch slot forever.
    except Exception as err:  # pylint: disable=broad-exception-caught
        expected = isinstance(err, TaskError)       # a refused message needs no traceback
        if kind is None:
            logger.warning("delivery %s refused: %s", tag, err, exc_info=not expected)
        else:
            logger.error("delivery %s: %s failed and was rolled back: %s: %s", tag, kind,
                         type(err).__name__, err, exc_info=not expected)
        return self._settle(channel, tag, ack=False)
    logger.info("delivery %s: %s committed in %.1f s: %s", tag, kind,
                time.monotonic() - started, summary)
    return self._settle(channel, tag, ack=True)
```

- The JSON is parsed and the kind is looked up in the task map TASKS = {"scrape_new_data": handle_scrape_new_data, "recompute_analytics": handle_recompute_analytics}.
- The handler runs inside one database transaction per message (with conn.transaction():), logged in as the worker role. basic_ack is sent only after the commit succeeded.
- Any failure rolls the transaction back and sends basic_nack(requeue=False): the message is logged and dropped, never requeued, so a bad task cannot loop forever and block the queue. That covers a body that is not JSON, JSON that is not an object, an unknown kind, a bad payload, a body over 64 KiB, JSON nested too deeply to parse, and any error in a handler.
- Each task runs in its own thread, so pika keeps answering broker heartbeats during a long pull. If the broker connection is lost, the process exits and Docker restarts it; RabbitMQ then redelivers the unacknowledged task, which is safe because both handlers are idempotent.

handle_scrape_new_data(conn, payload) reads the watermark first: ingestion_watermarks.last_seen for the source gradcafe, or payload["since"] when it is given, or, before the first pull, the highest p_id already stored. It runs my existing scraper through src/worker/etl/incremental_scraper.py, which fetches only entries with a higher result id and keeps the robots.txt check, the pause between pages and the 50-page cap. The new entries are cleaned, inserted with ON CONFLICT (p_id) DO NOTHING, and the watermark is moved to the highest result id seen; it never moves backwards. Finally the analysis snapshot is recomputed, so the page shows the new rows. All of this happens in the message's one transaction.

handle_recompute_analytics(conn, payload) recomputes all eleven analysis answers with SQL (src/worker/etl/query_data.py) and stores them in analysis_snapshot, in the same per-message transaction. The web page reads only that row, so the web app never runs the analysis itself.

8. Verification

I ran every check below on my own machine with the stack from section 3.

Check	Result
docker compose up --build	db, rabbitmq and web healthy, worker up, init exited 0 after loading 30,500 rows (section 3)
http://localhost:8080 and http://localhost:15672	both reachable (figures 1 and 2)
Update Analysis	202, queued banner; the worker logged committed, then acknowledged (basic_ack) after the commit; the page showed the new time
Message waits while the worker is stopped	Ready 1, then 0 after the worker restarted (figures 5 and 6)
Pull Data, first click	scraped 2 pages, inserted 21 rows; started after the newest stored entry 1020481 (no watermark row yet), watermark set to 1020502; the page shows 30,521 entries
Pull Data, second click	1 page read, inserted 0, watermark unchanged (idempotent)
Images on Docker Hub	web-v1 and worker-v1 public; anonymous pull works (figure 3)

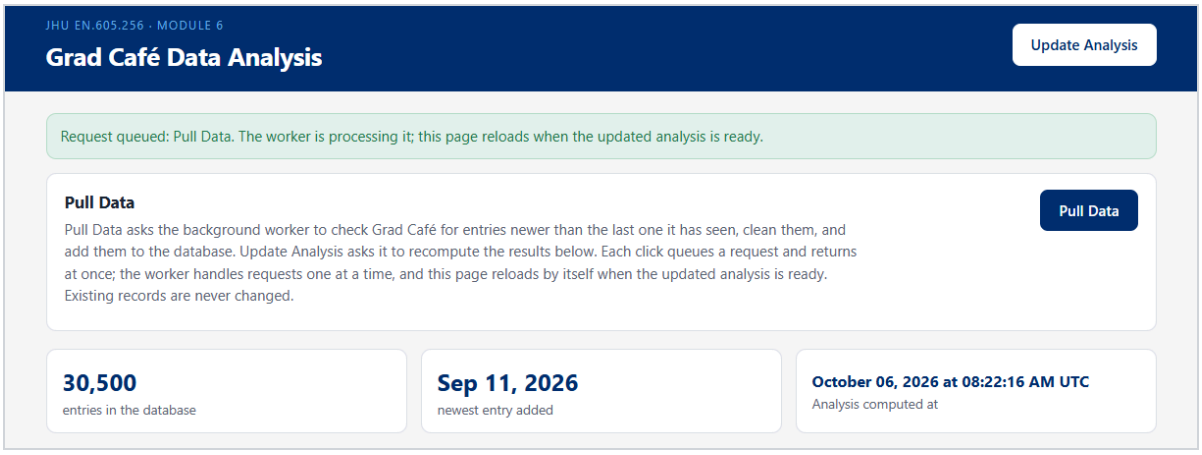


Figure 7. Pull Data answered 202 and the request was queued.

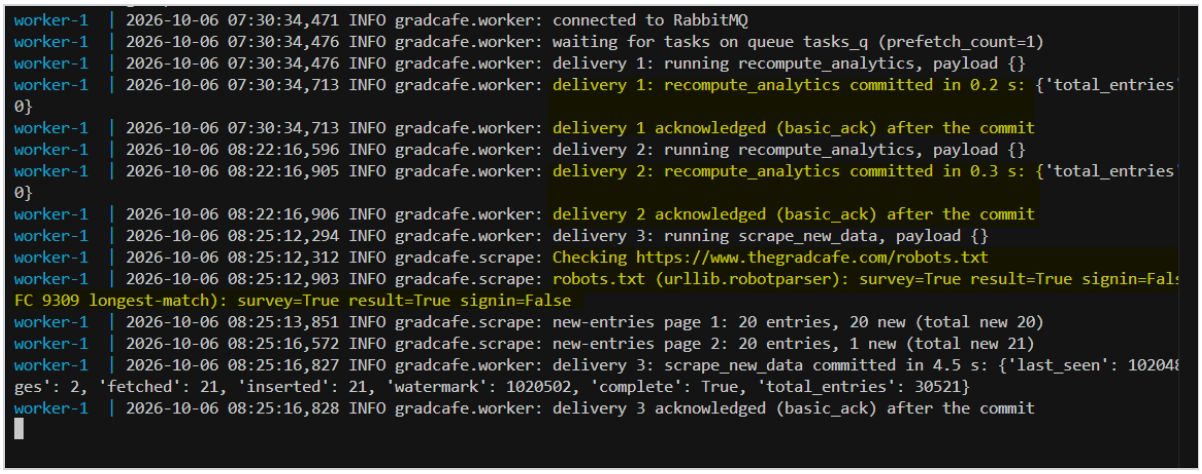


Figure 8. The worker log: each task commits, then is acknowledged. Delivery 3 is the first pull: 21 new entries inserted and the watermark set to 1020502.

The second pull, from the same log, found nothing new:

```
delivery 4: running scrape_new_data, payload {}
new-entries page 1: 20 entries, 0 new (total new 0)
delivery 4: scrape_new_data committed in 1.4 s: {'last_seen': 1020502, 'pages': 1, 'fetched': 0,
    'inserted': 0, 'watermark': 1020502, 'complete': True, 'total_entries': 30521}
delivery 4 acknowledged (basic_ack) after the commit
```

The watermark in the database matches the newest stored entry:

```
docker compose exec db psql -U gradcafe -d gradcafe \
  -c "SELECT source, last_seen, updated_at FROM ingestion_watermarks;" \
  -c "SELECT count(*), max(p_id) FROM applicants;"

  source  | last_seen |          updated_at
----------+-----------+-------------------------------
 gradcafe | 1020502   | 2026-10-06 08:29:52.209239+00

 count |   max
-------+---------
 30521 | 1020502
```

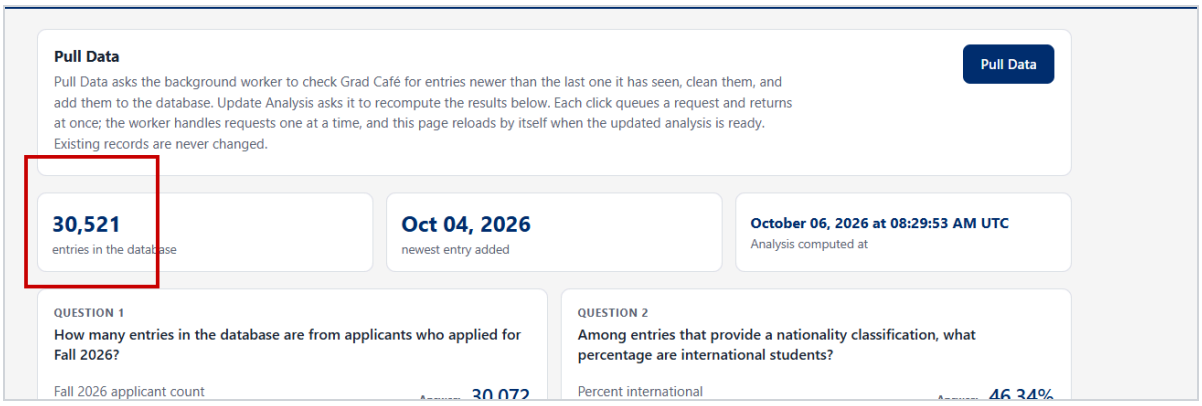


Figure 9. The page after the worker finished the pull: 30,521 entries, a newer newest entry and a new analysis time. The page reloaded by itself.

9. Tests, Pylint and CI

Pylint scores 10.00/10 on module_6/src with its default settings. The 528 tests cover 100% of module_6/src and need no RabbitMQ, Docker or internet: pika's connection, the broker and Grad Café are faked, and the database tests use a separate *_test database. They check, among other things, that the ack is sent only after the commit, that a failing task is rolled back and nacked without requeue, that prefetch_count is 1, that every declaration is durable, that messages are persistent, that the watermark advances and a second pull inserts 0, and that each database role has exactly its privileges.

.github/workflows/module_6.yml runs on every push that changes module_6/ with three jobs: Pylint (--fail-under=10), Pytest (PostgreSQL 17 service, 100% coverage required) and a Docker build of both images. Its first run passed all three in 55 seconds. The Module 5 and Module 4 workflows only run for their own folders, so they never interfere with Module 6.

10. Limitations

- A pull reads at most 50 listing pages. If more new entries than that ever pile up, the watermark still moves to the newest id and the older part of the gap is skipped (the worker logs a warning).
- Rows added by a pull have empty LLM columns until the optional LLM standardizer from the earlier modules is run.
- web, worker and the management UI share one RabbitMQ account; separate publish and consume users would need a RabbitMQ definitions file.
- The stack runs one worker: the queue and prefetch_count=1 serialize the tasks, and scaling to several workers is not supported.